

Multi-Agent Systems in the Wild: A Code-Grounded Empirical Study of Open-Source Agent Repositories

HKUST UROP 1100 Report

Author: Arnav Bala

Supervisor: Professor Arpit Narechania

May 13, 2026

Contents

1	Introduction	3
2	Project Status	4
2.1	What is finished	4
2.2	Future corpus expansion and improvement opportunities	4
3	Method Overview	5
3.1	Step 1: Discover candidate repositories	5
3.2	Step 2: Build a balanced sample	5
3.3	Step 3: Analyze repository code	5
3.4	Step 4: Aggregate and reconcile	6
4	Current Dataset Snapshot	6
4.1	Reconciliation details	6
5	How MAS Is Being Used in Practice	7

6	Most Used MAS Libraries and Frameworks	8
7	Implementation Overhead by Orchestration Pattern	8
8	Discussion	9
9	Limitations	9
10	Future Improvement Directions	10
11	Conclusion	10
A	Reference repositories used in examples	11

Abstract

This research report examines how open-source repositories labeled as multi-agent systems actually behave in code. We built a reproducible pipeline that discovers candidate projects, applies stratified sampling, performs shallow cloning, and generates structured repository analyses. The current snapshot contains 459 completed Markdown reports and 517 CSV entries, with 446 fully classified records. Results show a consistent gap between repository marketing language and runtime coordination semantics: many projects are agent-related but do not execute true multi-agent collaboration. Among repositories that do, workflow automation patterns dominate. Additional scans indicate frequent mentions of MCP, retrieval stacks, and browser or shell tooling, suggesting integration surface is a major practical differentiator in real deployments.

1 Introduction

The central goal of this research is to understand how multi-agent systems are actually being used in real open-source projects. This direction was inspired in part by early research notes from Yanwei Huang, a Ph.D. student supervised by Professor Arpit Narechania, who highlighted practical gaps around parallelism and coordination in current MAS design tools.

The core challenge is that repository labels do not always match runtime behavior. A project may describe itself as *multi-agent*, *agentic*, or *autonomous*, but the implementation may still be a single-agent flow or standard automation pipeline. The opposite also happens: some repositories with modest descriptions contain real multi-agent coordination in code.

To address this gap, this project uses a repeatable, code-grounded workflow. Instead of depending only on README claims, we inspect source files, produce structured repository reports, and aggregate the findings in a classification CSV. This approach lets us measure not only how often agent language appears, but how multi-agent systems are used in practice.

Concretely, this report delivers three outputs: a code-grounded usage profile of MAS repositories, a ranked view of the most visible framework ecosystems, and a practical analysis of what extra code is needed to implement different orchestration patterns.

This document is written as a **research report**. The pipeline produced a large snapshot, with

some known data-quality gaps in metadata consistency and incomplete rows in the aggregate CSV.

2 Project Status

2.1 What is finished

The core workflow is complete and stable enough for large-batch runs:

- Repository discovery and filtering are implemented in `mas_scraper.py`.
- Balanced sampling across use-case buckets and popularity tiers is implemented in the `build-balanced-sample` script.
- Automated repository analysis is implemented in `analyze_repos.py`.
- Structured outputs are saved in `repo_reports/*.md`.
- Aggregate labels are tracked in `repo_reports/_classifications.csv`.

2.2 Future corpus expansion and improvement opportunities

If this work is extended in the future, four practical improvements would add value:

1. Analyze more repositories so the dataset better covers fast-moving and newly updated projects.
2. Resolve incomplete or blank classification rows caused by failed runs and parsing drift.
3. Run one final consistency pass so report counts and CSV counts match the same snapshot date.
4. Add a short manual audit sample to estimate labeling reliability.

These are optional future improvements and do not require a redesign of the pipeline.

3 Method Overview

3.1 Step 1: Discover candidate repositories

We query GitHub using topic tags and keyword phrases that are common in agent projects. We then apply practical filters: minimum stars, minimum contributors, non-empty README, and recent activity. This helps remove abandoned or low-signal repositories while keeping a broad set of projects.

3.2 Step 2: Build a balanced sample

Discovered repositories are grouped into five heuristic use-case buckets:

- Workflow Automation
- Code Generation
- RAG + Agents
- Browser / Terminal Use
- Simulation

Each repository is also placed in a popularity tier (Mainstream, Mid-Tier, Niche). Sampling is balanced across these groups so one category does not dominate the study.

3.3 Step 3: Analyze repository code

For each sampled repository, we shallow-clone the codebase and run a structured analysis prompt. The generated report is not only descriptive. It also ends with three machine-readable fields:

- `mas_related`
- `uses_mas`
- `use_case`

These fields are extracted into a CSV for aggregation.

3.4 Step 4: Aggregate and reconcile

Finally, we compare three counts: Markdown reports, CSV rows, and fully populated classifications. Differences between these counts are expected in a long-running pipeline, but they should be explained and reduced before final submission.

4 Current Dataset Snapshot

Table 1 gives the main status numbers for this research report.

Metric	Count
Completed Markdown reports	459
Rows in classification CSV	517
Rows with full classification fields	446
Rows still incomplete	71

Table 1: Current project snapshot used in this research report.

Two gaps are important. First, there are more CSV rows than Markdown reports. This usually happens when repositories are tracked by the ledger but analysis fails (for example, clone limits or timeouts). Second, some Markdown reports still do not map cleanly to a complete classification row because of parsing and backfill drift. Both gaps are documented here and represent clear opportunities for future cleanup.

4.1 Reconciliation details

To make the counts transparent, the snapshot can be read in two short equations:

- $517 - 459 = 58$: repositories tracked in the ledger/CSV but without a completed Markdown report in the current snapshot.
- $459 - 446 = 13$: repositories with a Markdown report that still do not have a complete classification triple in the CSV.

These values are consistent with the 71 incomplete CSV rows shown in Table 1.

Table 2 shows how completed reports are distributed by upstream sampling bucket.

Upstream bucket	Completed reports
Browser / Terminal Use	111
Code Generation	99
Simulation	89
RAG + Agents	89
Workflow Automation	71

Table 2: Completed reports by upstream heuristic bucket.

The bucket totals show that the sample remains broad. No single bucket overwhelms the dataset.

5 How MAS Is Being Used in Practice

Table 3 summarizes how the 446 fully classified repositories are used in code. The strongest pattern is straightforward: workflow-oriented implementations dominate the final labels, whether or not a repository runs coordinated multi-agent behavior at runtime.

Code-grounded outcome ($n = 446$)	Count
uses_mas=yes	213
uses_mas=no	233
use_case=Workflow Automation	348
use_case=None	78
use_case=Simulation	11
use_case=RAG + Agents	7
use_case=Code Generation	2

Table 3: How MAS-related repositories are used in practice after code-grounded classification.

The most frequent joint outcomes are also informative:

- mas_related=yes, uses_mas=yes, use_case=Workflow Automation: 208 repositories.
- mas_related=yes, uses_mas=no, use_case=Workflow Automation: 139 repositories.
- mas_related=no, uses_mas=no, use_case=None: 78 repositories.

Taken together, these counts suggest that many repositories in this ecosystem are built around predictable orchestration pipelines, and only a subset implements explicit multi-agent coordination loops.

6 Most Used MAS Libraries and Frameworks

To answer which libraries appear most often, we counted the `architecture_labels` field in frontmatter across 459 completed reports. Table 4 shows the ranking.

Architecture label frequency ($n = 459$ reports)	Count
CrewAI	382
LangGraph	96
LangChain	88
Custom/Other	74
AutoGen	57

Table 4: Most frequent framework labels in report frontmatter.

These are prevalence signals, not strict import-level ground truth. Still, they provide a useful answer to which ecosystem names are most visible in current open-source MAS usage.

7 Implementation Overhead by Orchestration Pattern

Beyond naming frameworks, a practical question is what extra code is needed to make different orchestration patterns work. Table 5 summarizes this using motif prevalence from report §3 text scans (Hierarchical 235, Graph-based 215, Event-driven 128, Swarm/P2P 112, Blackboard 5) plus code-grounded interpretation.

This table is descriptive for this corpus snapshot. It suggests that moving from linear workflow orchestration to decentralized handoff patterns typically adds more supporting code around routing, state management, and operational safety.

Concrete examples from analyzed repositories. The pattern-to-overhead mapping in Table 5 is also reflected in repository-level evidence: `actions/cache` illustrates low-overhead deterministic workflow steps; `openai/swarm` shows explicit handoff logic for higher-overhead

Pattern family	Typical overhead	Extra code usually required
Workflow / linear pipelines	Low–Medium	Task sequencing, tool wrappers, retry/error handling, logging, and state passing between steps.
Hierarchical manager-worker	Medium	Router or supervisor agent logic, role-specific prompts, handoff contracts, and worker result aggregation.
Graph / state-machine orchestration	Medium–High	Node-edge definitions, conditional transitions, shared state store, guardrails for loops, and traceability hooks.
Swarm / peer-to-peer handoff	High	Dynamic routing, agent-selection policies, stronger memory/context control, conflict handling, and observability for non-linear paths.
Event-driven / bus-based patterns	High	Message schemas, queue or bus adapters, idempotency controls, retries/dead-letter handling, and consumer coordination.

Table 5: Implementation overhead needed to operationalize common orchestration patterns.

peer transfer; `langchain-ai/langchain` demonstrates graph/state-machine style routing primitives; `microsoft/autogen` highlights event/message-oriented coordination infrastructure; and `HKUDS/LightRAG` provides explicit multi-agent transition maps in demo flows.

8 Discussion

The report does not argue that multi-agent systems are rare. Instead, it shows that public labels and practical usage are not always the same. When code-level evidence is required, workflow-heavy implementation patterns dominate, while explicit multi-agent coordination appears in a smaller but still substantial share of projects.

For research and benchmarking, this matters because conclusions can shift depending on whether repositories are counted by keywords or by runtime behavior in code.

9 Limitations

This research report should be read with the following limits in mind:

- The analysis relies heavily on one automated reviewer workflow.
- Repositories are shallow-cloned, so deep history is not analyzed.
- Very large repositories may be skipped for compute and storage reasons.
- Timeouts can truncate difficult repositories.
- Keyword scans over report text are useful but still approximate.

None of these limits invalidate the project, but they explain important scope and reliability boundaries for this research report.

10 Future Improvement Directions

If this dataset is extended later, the following quality-focused steps would be most useful:

1. **CSV cleanup pass:** regenerate and reconcile incomplete rows using existing reports and state records.
2. **Manual validation sample:** hand-check a stratified subset to estimate agreement with automated labels.
3. **Stability rerun:** rerun a smaller batch to confirm that counts remain consistent on a fresh snapshot.
4. **Final write-up polish:** tighten wording, align all tables with one snapshot date, and lock the final appendix list.

Completing these steps in a future extension would further improve reliability.

11 Conclusion

This research report presents a working, reproducible pipeline for studying open-source multi-agent repositories at scale. The current snapshot is already large enough to support meaningful

observations. The most important result is a practical one: repository labels and real runtime behavior are often different, so code-grounded analysis is necessary for trustworthy conclusions.

The key takeaway is not architectural change but quality consolidation: reconciling incomplete rows, adding a targeted manual audit, and freezing one clean snapshot would make future releases more robust.

A Reference repositories used in examples

- [actions/cache](#)
- [HKUDS/LightRAG](#)
- [langchain-ai/langchain](#)
- [microsoft/autogen](#)
- [openai/swarm](#)